

Model Training for Churn Risk Prediction Workflow Documentation

This document distills the core mindset and systematic approach applied in the *Reprocessing_Data.ipynb* script. It emphasizes key decisions and the overarching methodology rather than specific code details.

1. Data Understanding

- **Objective Clarity:** Begin by identifying the prediction goal—discrete churn risk levels (1–5).
 - **Schema Inspection:** Rapidly survey column types, distributions, and missing-value patterns to guide cleaning strategies.
-

2. Data Cleaning and Consistency

- **Irrelevant Feature Removal:** Exclude identifiers and non-predictive columns (IDs, names, timestamps) to reduce noise.
 - **Missing-Value Standardization:** Convert varied placeholders (e.g., ?, unknown, -1) into a single missing-data marker to simplify imputation logic.
 - **Target Validation:** Isolate and remove any records lacking a valid churn score, ensuring only reliable labels drive model training.
-

3. Feature Engineering

- **Temporal Features:** Derive experience-based measures (e.g., years since joining) from raw dates, capturing temporal dynamics without retaining raw timestamps.
- **Domain-Informed Mapping:**
 - Collapse detailed categories into ordinal or binary scales based on business significance (membership tiers, complaint statuses, feedback sentiment).
 - Leverage subject-matter knowledge to bucket free-text or multi-valued fields into meaningful groups (e.g., 'good' vs. 'bad' feedback).

4. Stratified Data Splitting

- **Preserving Class Balance:** Use stratified sampling to maintain churn-score proportions across training and test sets, preventing skewed evaluation.
 - **Model Compatibility:** Adjust label indexing only where mandated by modeling libraries, keeping the core dataset unchanged.
-

5. Dynamic Preprocessing Pipelines

- **Automated Column Discovery:** Programmatically identify numerical, binary, and nominal features to minimize manual interventions and ensure reproducibility.
 - **Three-Tier Pipeline Design:**
 1. **Numeric Pipeline:** Impute missing values with central tendencies and scale to a common range for algorithmic stability.
 2. **Binary Pipeline:** Impute and apply compact encoding to two-valued features, preserving interpretability.
 3. **Nominal Pipeline:** Impute and one-hot encode multi-class features, enabling the model to learn categorical distinctions.
 - **Unified Transformation:** Combine these sub-pipelines into a single transformer, guaranteeing consistent application during both training and inference.
-

6. Model Integration and Training

- **Model Benchmarking:** Evaluated a suite of algorithms—including Random Forest, Bagging, Gradient Boosting, and K-Nearest Neighbors—to assess baseline performance and overfitting tendencies. XGBoost was selected for its superior generalization across validation folds.
- **Encapsulated Workflow:** Embed preprocessing and the XGBoost classifier into a single pipeline, avoiding manual pre- and post-processing steps.
- **Objective Selection:** Configure the model for multi-class classification directly on original churn score levels to align with business requirements.
- **Encapsulated Workflow:** Embed preprocessing and the XGBoost classifier into a single pipeline, avoiding manual pre- and post-processing steps.

- **Objective Selection:** Configure the model for multi-class classification directly on original churn score levels to align with business requirements.
-

7. Efficient Hyperparameter Optimization

- **Focused Search Space:** Limit tuning to the most influential parameters (learning rate, tree depth, sampling ratios, tree count, pruning thresholds).
 - **Random Sampling Strategy:** Employ randomized search with a controlled number of iterations, balancing computational budget against discovery of high-performance configurations.
 - **Cross-Validation Rigor:** Use stratified folds and macro-averaged F1 scoring to ensure robust performance across all classes, including less frequent categories.
-

8. Evaluation and Interpretation

- **Comprehensive Metrics:** Assess accuracy alongside precision, recall, and F1 scores for each class, with weighted or macro averaging to contextualize class imbalances.
 - **Overfitting Check:** Compare training vs. test performance to detect and mitigate model overfitting, informing further regularization or feature selection.
-

9. Model Persistence

- **End-to-End Serialization:** Save the full pipeline—including data transformations and the trained estimator—to ensure consistent, reproducible inference in downstream applications.
 - **Version Documentation:** Record model parameters, performance metrics, and code environment details to facilitate traceability and future audits.
-

Conclusion This methodology emphasizes clarity of purpose, automated yet transparent preprocessing, and efficient model tuning. By structuring the workflow around these mental models, we ensure scalability, reproducibility, and alignment with business objectives in predicting customer churn risk.